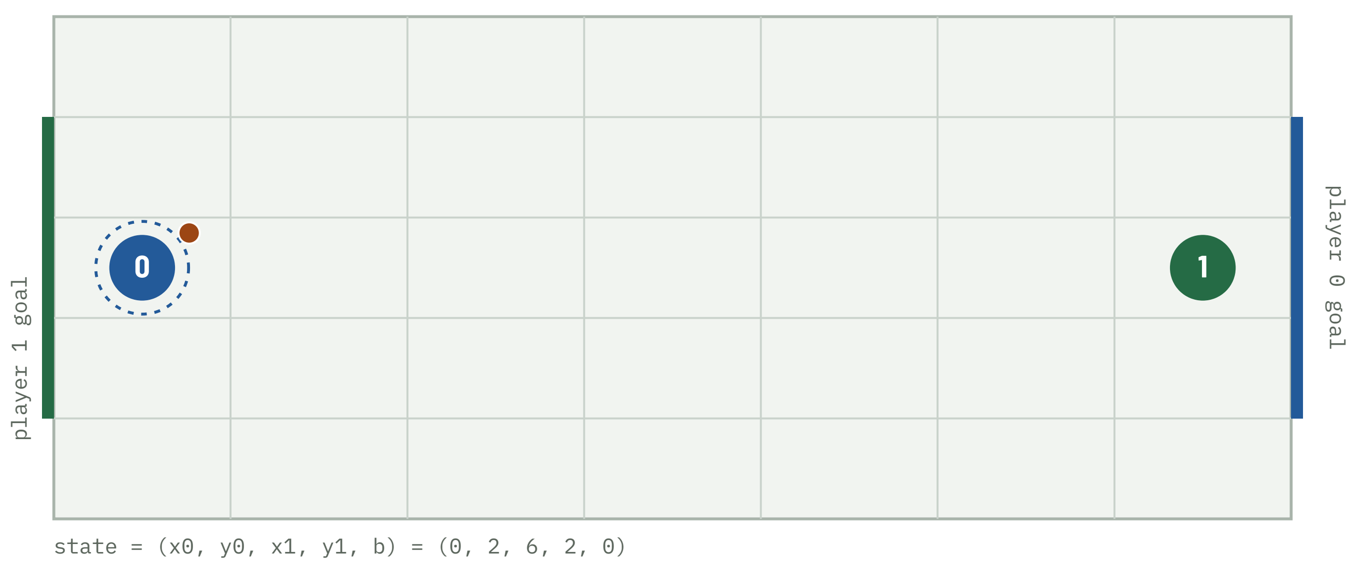
When Does Soccer Need Mixed Strategies?

A pure-first Nash-Q approach: where a two-player soccer Markov game does — and does not — need mixing, and how much LP work that saves.

> FOUR QUESTIONS
>
> **RQ1 Existence** — under what transition structures does a pure equilibrium exist at every state? **RQ2 Efficiency** — how much work does checking for a pure saddle first eliminate? **RQ3 Mechanism** — which spatial configurations force mixing? **RQ4 Robustness** — how do discounting and tolerance affect the split?

`repo · soccer-markov-nash | 150+ tests | tables from experiments/*.csv | scope in docs/assumptions.md`



The kickoff. 7×5 grid, 2 380 non-terminal states. Player 0 (blue) carries the ball and attacks the right edge; player 1 (green) the left. Actions U D L R are chosen simultaneously; reward is +1 / −1 / 0, zero-sum.

ABSTRACT

1 **For the implemented deterministic A10 model, every one of the 2 380 converged stage games has a pure saddle.** Playing a pure saddle action everywhere is then a stationary pure-strategy equilibrium, found in 9 sweeps with no linear program.

2 **A coin-flip tie-break does not change this;** Littman's random *move order* does — but only when the goal mouth is more than one cell wide, and even then on 3.95% of states (this configuration).

3 **Those mixed states are geometrically localised:** a decision tree predicts them from the state with precision 0.96, and 68 of 94 have a 2×2 matching-pennies support.

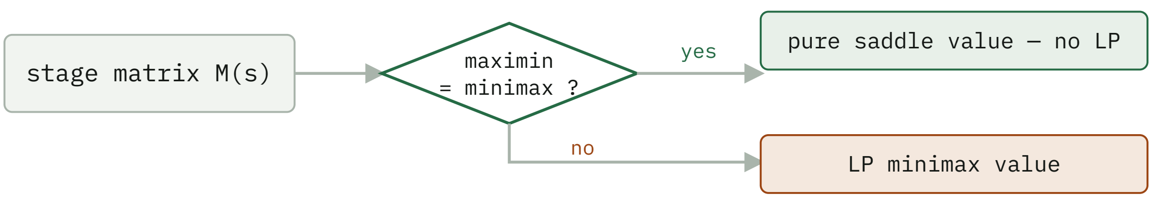
4 **The pure-first hybrid solver** matches the all-LP value function to 4e−16 while calling the LP on only 3.95% of states — ~25× fewer per sweep; mirror symmetry halves the work again.

Method

The game is zero-sum, so at every state the stage game is a payoff matrix $M(s)[a_0, a_1] = E[R + \gamma V(s')]$ and the Shapley (1953) fixed point is the minimax value:

$$
\begin{aligned}
V(s) = \mathrm{val}(M(s)) &= \max_n \min_{k3} \, (p^\top M)[a_1] \\
&= \min_p \max_{a0} \, (Mq)[a_0]
\end{aligned}
$$

The $Q = R + \beta\,\mathrm{Nash}(Q')$ update is this operator. Three stage backups: **pure** (the maximin security value — a fast lower bound, not a Nash value when it is below the minimax), **mixed** (the LP minimax value, via `scipy` HiGHS), and **hybrid** (the pure saddle value where `maximin = minimax`, the LP elsewhere — exact everywhere). Support enumeration (`support_enum.py`) finds *all* equilibria, including of general-sum games.



The pure-first hybrid. On the deterministic game the "yes" branch is taken at every one of the 2 380 states, so the LP is never called.

RQ1 — Existence

MOVE ORDER	STOCHASTIC	CONVERGED STAGE GAMES WITH NO PURE SADDLE	STATIONARY PURE EQ?
deterministic (exact A10)	no	0 / 2380	yes (Claim B)
coinflip tie-break	yes	0 / 2380	yes (Claim B)
random move order	yes	94 / 2380 (this config)	no

IT IS THE MOVE ORDER, NOT THE RANDOMNESS

The deterministic and coin-flip value functions are *identical* (`max |V_det − V_coin| = 0` at every γ): optimal play never enters a losing contest, so the coin is never flipped. Only Littman's random move order — where a ball-steal depends on who resolves first *and* on both players' targets — creates matching-pennies subgames.

RQ2 — Efficiency

Two numbers, kept apart: the **LP-call rate** is a property of the game and reproduces exactly; **wall clock** is a median of 5 repeats and depends on the machine and LP backend (`scripts/benchmark.py`).

RANDOM MOVE ORDER, Γ = 0.9 — THE CASE WHERE THE LP IS ACTUALLY NEEDED

SOLVER	SWEEPS	LP CALLS	STATES NEEDING LP	MEDIAN WALL CLOCK
`pure (lower bound)`	18	0	—	0.3 s
`hybrid exact`	108	9 672	3.95%	13.2 s
`all-LP exact`	108	~3.6 M	100%	~8 min

The hybrid solves an LP on only the **3.95%** of states lacking a pure saddle — a ~25× per-sweep reduction — and matches the all-LP value to `4e−16`. The wall-clock ratio (~13 s vs ~8 min) is larger but less portable: it also reflects matrix construction and the LP backend. **Mirror symmetry** (`run_symmetric`, deterministic dynamics only): every state has a distinct board-flip-plus-player-swap mirror, so the 2 380 states are 1 190 pairs — reconstruct one from the other by `V(mirror(s)) = −V(s)`, cutting the deterministic solve 0.33 s → 0.23 s.

Freeze-then-iterate, and its staleness

RANDOM MOVE ORDER, HYBRID SOLVER — SAME FIXED POINT, |DIFF| < 3E−9

	ROUNDS / SWEEPS	MATRIX-GAME SOLVES	WALL CLOCK
`value iteration`	108	9 672	13.8 s
`policy iteration`	21	1 879	17.2 s

5× fewer LP solves — but the frozen strategies' distance to the current Nash stays *maximal* (a full pure-strategy flip) for **16 outer rounds**, then collapses to `< 1e−8`. The thrashing eats the saving; on the deterministic game it is strictly worse. Value iteration with the per-sweep Nash cache wins here.

RQ3 — Mechanism

Goal-mouth width, not board size, is the gate. The phase diagram sweeps every board with `width ≤ 11`, `height ≤ 9`, `width·height ≤ 45` against goal widths 1 to `height−2` (`scripts/phase_diagram.py --analyze`, `experiments/phase_diagram.csv`).

	goal–mouth width						
	1	2	3	4	5	6	7
3x3	0						
3x4	0	12					
3x5	0	10	9				
3x6	0	7	7	7			
3x7	0	7	6	6	6		
3x8	0	6	6	5	5	5	
3x9	0	6	5	5	4	4	4
4x3	0						
4x4	0	8					
4x5	0	8	6				
4x6	0	7	6	5			
4x7	0	7	5	5	4		
4x8	0	6	5	5	4	4	
4x9	0	6	5	4	4	4	3
5x3	0						
5x4	0	7					
5x5	0	6	4				
5x6	0	6	4	4			
5x7	0	5	4	4	3		
5x8	0	5	4	3	3	3	
5x9	0	5	4	3	3	3	3
6x3	0						
6x4	0	8					
6x5	0	6	4				
6x6	0	5	4	3			
6x7	0	4	4	4	3		
7x3	0						
7x4	0	7					

7x5	0	5	4	
7x6	0	5	4	3
8x3	0			
8x4	0	6		
8x5	0	5	4	
9x3	0			
9x4	0	5		
9x5	0	4	3	
10x3	0			
10x4	0	4		
11x3	0			
11x4	0	4		

cells = mixed-state %

The gate. Goal width 1 → **0** mixed states, on all 40 one-cell configs. Goal width ≥ 2 → mixing on all 87: whether a board mixes is a *perfect* function of goal width. The *fraction* (2.7–12%) is an OLS R^2 0.64 in board area — a dilution effect — with goal width adding little and a negative coefficient. Every state is reachable from the kickoff.

Geometry predicts the label (`scripts/geometry_model.py`): a depth-4 decision tree on carrier-frame features separates `mixed` from the rest with **precision 0.96, recall 0.91**. The dominant feature is `defender_can_intercept` — the defender within one move of the carrier's forward cell (`P(mixed) = 0.44` vs `0.002`).

Beyond prediction, the 94 mixed states canonicalize under the board mirror to 47 pairs and cluster into **8 geometric templates** (`scripts/templates.py`, `docs/templates.md`).

THE MIXED-STATE TEMPLATES — CARRIER-FRAME GEOMETRY, EQUILIBRIUM SUPPORT

STATES	GEOMETRY	SUPPORT	STRUCTURE
24	defender 1 ahead, carrier one row off the goal row	2×2	matching pennies
24	defender 1 ahead, same row, carrier on a goal row	2×2	matching pennies
16	defender 2 ahead, same row, carrier on a goal row	2×2	matching pennies
4	defender diagonally adjacent, carrier off the goal row	2×2	matching pennies
14	defender 1–2 ahead, same row, on a goal row	2×1	near-pure saddle
8	defender 2 ahead, same row, on a goal row	3×2	degenerate (row ties)
4	defender 2 ahead, carrier off the goal row	3×3	wider mix

The **four 2×2-support templates are all verified matching pennies** — the carrier's best reply flips between the two defender columns and vice versa — covering **68 of 94** states: carrier {climb, advance} against defender {cover a lane, hold the forward cell}. All 94 share one value across every equilibrium; 64 have a unique one.

Why the mix is unavoidable. A mixed stage game needs all three of: a stochastic (½–½) resolution order; a goal mouth wider than one cell, so the carrier has two scoring lanes; and a defender close enough to contest the forward cell but unable to cover both lanes in one move. The carrier climbs or drops, the defender guesses which — matching pennies. Drop any one clause — a single goal cell, deterministic resolution, or the coin-flip tie-break — and every stage game has a pure saddle. The full argument and one stage matrix per template are in `docs/templates.md`. For the single goal cell the theorem is partly proved (`docs/proof.md`): the defender has a closed-form optimal strategy — guard the one cell — and every stage game is dominance-solvable, machine-checked for all boards up to 11×5.

RQ4 — Numerical robustness

The value is three numbers

`value_bracket` returns what the row player guarantees ($\min_k (pM)_k$), gets ($p^\top Mq$), and can reach ($\max_i (Mq)_i$). The true value is bracketed, so the width certifies the LP error.

> FINDING
>
> With `scipy`'s HiGHS the three agree to `1.1e−16` per stage game, `8.7e−10` accumulated. The concern is real for older vertex/simplex solvers — a non-issue here.

Discounting and tolerance

	DETERMINISTIC	RANDOM (7×5)
mixed states, γ = 0.5 → 0.9 → 0.995	0 → 0 → 0	122 → 94 → 102
classification split, rel_tol 1e−12 → 1e−3	stable	604 / 1682 / 94 (stable)
classification, rel_tol 1e−2 / 1e−1	—	80 / 0 mixed
fixed 0.1-rounding flips a saddle	0	62

Discounting mildly shifts which states mix; the classification is otherwise robust across nine orders of magnitude of tolerance, breaking only as the tolerance approaches the real `minimax − maximin` gaps. Fixed 0.1-rounding — proposed to force indifference — misclassifies 62 states, exactly the small-entry mixed region.

Secondary validation — the policy plays correctly

- Duality gap `V0_br(s₀) + V1_br(s₀) < 1e-9` for every move order — no opponent beats the game value.
- Nash vs. Nash reproduces the value: forced draws in the deterministic and coin-flip games; `+0.149 ± 0.005` empirical discounted return over 5 seeds of 3000 games (vs. `+0.150` computed) in the random game.
- A best response to one assumed opponent is fragile: the Part 2 policy has exploitability `0.43` — worse than random — which is the A10 competition's warning about non-equilibrium submissions.
- A10 networks over 5 seeds (`experiments/a10_competition_seeds.csv`): the Part 2 imitation net plays optimally at `0.990 ± 0.000` of states and wins Q8 every time; competition Network Second is exact, but Network First sits at a stubborn `0.28 ± 0.01` exploitability that does not wash out with re-seeding.

Beyond zero-sum, and open questions

- **General-sum — extension point.** `markov_game.py` solves 2-player general-sum Markov games by enumerating *all* stage equilibria (`support_enum.py`) and selecting one — the notes' "largest sum of values", a no-op for zero-sum but decisive for Battle of the Sexes, where it avoids the mixed equilibrium whose value is below either pure one. Pure-first throughout. The soccer game is zero-sum, so this does not touch the main result.
- **Function approximation — partial.** A network fit to the stage matrices (`nash_dqn.py`, 5→96→96→16 with biases, 600 epochs) trails the exact solver, over **5 seeds** (`experiments/nash_dqn_seeds.csv`):

	EXACT HYBRID NASH-Q	NEURAL NASH-Q (MEAN ± SD)
max |V − V_exact|	0	0.55 ± 0.02
mean |V − V_exact|	0	0.13 ± 0.00
action agreement	100%	43% ± 2%
pure/mixed classification	100%	67% ± 2%
exploitability	<1e-9	0.43 ± 0.04
convergence (epochs to plateau, of 600)	exact	469 ± 31; 3/5 still creeping
runtime	9 sweeps, 0.3 s	600 epochs, ~35 s

Deterministic game only — `train_nash_dqn` rejects the stochastic variants, so the network never has to represent a genuinely mixed stage game. Even so it trails: it fits the ~1 000 zero-value states easily (small *mean* error) but misses the γ^k bands and contested regions — worst-state error `0.55`, policy about as exploitable as random play, mis-calls *whether* a state needs mixing a third of the time. Keep the exact solver as ground truth.

- **A proof** (**partly done**). The single-cell pure-saddle theorem now has the defender's optimal strategy in closed form and a dominance-solvability certificate for every finite board (`docs/proof.md`); the carrier's half, board-size-free, is open.

Limitations

- The A10 page redacts the ID-specific start position and goal rows; this repo uses the standard Littman geometry (configurable). Across the phase-diagram sweep every one-cell-goal board has 0 mixed states and every wider-goal board has some; the exact `3.95%` is the 7×5 / 3-cell-goal case.
- Several collision sub-cases (carrier vs. stationary opponent, bump = steal, swap possession) are *interpreted* from the two stated A10 rules, not quoted. If course staff intended otherwise, Claim A must be re-measured. See `docs/assumptions.md`.
- **Claim A** (every converged stage game has a pure saddle) and **Claim B** (a stationary pure equilibrium exists) are established by enumeration. **Claim C** (the theoretical game necessarily has one, over all settings) is *not*, for the general goal mouth. For the single goal cell it is partly proved (`docs/proof.md`).
- `random` and `coinflip` are different game definitions, isolating the collision rule and the tie-break respectively — not the A10 game.

```
soccer_nash — game.py · matrix_games.py · support_enum.py · nash_q.py · markov_game.py ·
numerics.py · dominance.py · geometry.py · symmetry.py · tree.py · templates.py ·
onecell.py · reachability.py · best_response.py · exploit.py · mlp.py · nash_dqn.py ·
opponents.py · shaping.py · simulate.py · experiment.py · render.py
scripts — experiments.py · analyze.py · numerics.py · equilibria.py · geometry_model.py ·
templates.py · phase_diagram.py · onecell_proof.py · benchmark.py · policy_iteration.py ·
selfplay.py · nash_dqn.py · render.py · a10_part1.py · a10_part2.py · a10_competition.py
experiments — baseline.csv · gamma_sweep.csv · tolerance_sweep.csv · board_sweep.csv ·
goal_mouth_sweep.csv · one_cell_goal.csv · phase_diagram.csv · nash_dqn_seeds.csv
env: 7×5 grid, 2 380 states, zero-sum ±1, γ = 0.9
```